

module-04-lists

Chapter 4: Lists | 62 programs in learning order

Source: *Automate the Boring Stuff with Python 1e*, Al Sweigart (CC-BY-NC-SA). PPT Module-3: *create/index/slice/len/append/extend/insert/remove/pop/clear/index/count/sort/reverse/copy, +/*, in/not in, del*

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

A list holds many values in one variable: `spam = ['cat','bat','rat']`. Items are ordered, changeable, and indexed from 0 (or -1 from the end). Slices `spam[1:3]` grab a sub-list. for loops walk lists; `in/not in` test membership. Methods like `append/sort` act ON the list. Big trap: assigning a list copies the REFERENCE, not the items - use `copy.copy()/deepcopy()` for a real copy.

Key ideas to carry forward

- `spam[0]` first item; `spam[-1]` last; `spam[1:3]` items 1..2; `len(spam)` counts.
- `[1,2] + ['A']` concatenates; `['X'] * 3` replicates.
- `append(x)` end-add; `insert(i,x)` at position; `remove(v)` by value; `pop(i)` by index + returns it; `del spam[2]`.
- `index(v)` finds position; `count(v)` counts; `sort()/sort(reverse=True)` sorts in place; `reverse()` flips.
- Tuples `('a',1)` are immutable lists; `list()/tuple()` convert between them.
- Magic 8-Ball, Comma Code, and Character Grid are the capstone programs - read them last.

Classic beginner mistakes

- ! `IndexError` from `spam[10]` on a 3-item list; `sort()` on mixed numbers+strings fails.
- ! Appending to one variable and being surprised the 'other' list changed too (same reference).

Study order hint: 01-12 indexing/slicing, 13-21 loops + `in`, 22-44 methods, 45+ capstones (magic 8-ball, comma code, grid).

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. How is a list different from a tuple?

A: Lists are mutable (append/sort/in-place change); tuples are immutable and hashable, usable as dict keys. Convert with list()/tuple().

Q2. Explain negative indexing and slicing.

A: spam[-1] is the last item; spam[1:3] copies items 1..2 (stop excluded); spam[:] is a full copy.

Q3. append vs extend vs insert?

A: append(x) adds one item at end; extend(iterable) adds each element; insert(i, x) places at position i.

Q4. Why did my second list change when I edited the first?

A: Assignment copies the reference, not the items. Use copy.copy() (shallow) or copy.deepcopy() for nested lists.

Q5. How do you find, count, sort?

A: spam.index(v), spam.count(v), spam.sort() in place (reverse=True to flip), sorted(spam) for a new list.

Q6. in/not in and del - quick uses?

A: 'cat' in spam tests membership; del spam[2] removes by index; remove(v) removes by value; pop(i) removes and returns.

Teach it to others

The one analogy that sticks

A list is an egg carton: numbered slots (indexes), you can swap eggs (mutable), slice out a row (slicing). Copying the carton label (assignment) does not buy new eggs - both labels point at one carton (references).

What your students will get wrong

- x Indexes start at 1 (they start at 0).
- x `sort()` gives back a new list (it returns `None`; the list itself changes).
- x Copying a list with `=` duplicates it (it aliases it).

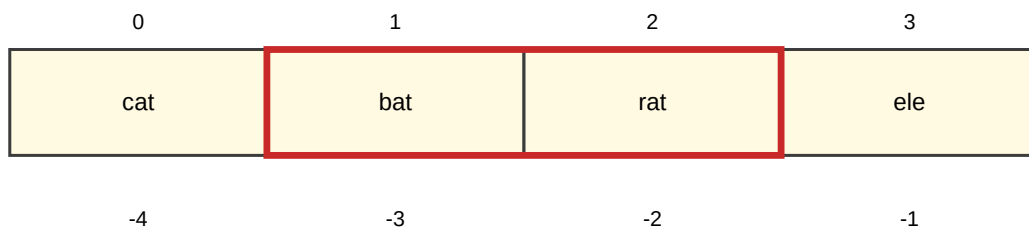
5-minute whiteboard plan

1. Draw the index strip for `['cat','bat','rat','ele']` with `-4..3` labels.
2. Live-demo the aliasing bug, then fix with `copy.deepcopy`.
3. Students predict `spam[1:3]` before running.
4. Exit ticket: remove 2nd item two ways (`del` vs `pop`) and state what each returns.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - index strip: 0-based on top, negative below



spam[0]='cat', spam[-1]='ele'. Red window = spam[1:3] (stop excluded). Slices copy; indexes point.

Visual 2 - assignment aliases, copy duplicates



b = a sticks two labels on ONE carton. Use copy/deepcopy for a second carton.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Guest list manager

```
guests = ['Al', 'Bo']
guests.append('Cy')
guests.insert(0, 'Zed')
print(guests)
print(guests.pop())
print(guests)
# EXPECTED: ['Zed', 'Al', 'Bo', 'Cy'] / Cy / ['Zed', 'Al', 'Bo']
```

X2. Grades report

```
scores = [88, 42, 95, 73]
print('avg:', sum(scores) / len(scores))
print('best:', max(scores))
scores.sort(reverse=True)
print(scores)
# EXPECTED: avg: 74.5 / best: 95 / [95, 88, 73, 42]
```

X3. Dedup keep order

```
seen = []
for x in [1, 2, 2, 3, 1, 4]:
    if x not in seen:
        seen.append(x)
print(seen)
# EXPECTED: [1, 2, 3, 4]
```

Program map - all 62 at a glance

```
01 01_list_data_type.py
02 02_getting_individual_values.py
03 03_getting_individual_values_spam.py
04 04_getting_individual_values_spam_2.py
05 05_getting_individual_values_spam_3.py
06 06_negative_indexes.py
07 07_getting_sublists_slices.py
08 08_getting_sublists_slices_spam.py
09 09_getting_list_s.py
10 10_changing_values_list.py
11 11_list_concatenation_list.py
12 12_removing_values_from.py
13 13_working_lists.py
14 14_working_lists_enter.py
15 15_working_lists_catnames.py
16 16_working_lists_enter_2.py
17 17_using_loops_lists.py
18 18_using_loops_lists_2.py
19 19_using_loops_lists_3.py
20 20_using_loops_lists_supplies.py
21 21_not_operators.py
22 22_not_operators_mypets.py
23 23_not_operators_enter.py
24 24_multiple_assignment_trick.py
25 25_multiple_assignment_trick_cat.py
26 26_multiple_assignment_trick_cat_2.py
27 27_multiple_assignment_trick_2.py
28 28_augmented_assignment_operators.py
29 29_augmented_assignment_operators_spam.py
30 30_augmented_assignment_operators_spam_2.py
31 31_augmented_assignment_operators_bacon.py
```

32 32_finding_value_list.py
33 33_finding_value_list_spam.py
34 34_adding_values_lists.py
35 35_adding_values_lists_spam.py
36 36_adding_values_lists_eggs.py
37 37_removing_values_from_spam.py
38 38_removing_values_from_spam_2.py
39 39_removing_values_from_spam_3.py
40 40_sorting_values_list.py
41 41_sorting_values_list_spam.py
42 42_sorting_values_list_spam_2.py
43 43_sorting_values_list_spam_3.py
44 44_sorting_values_list_spam_4.py
45 45_example_program_magic.py
46 46_list_like_types.py
47 47_mutable_immutable_data.py
48 48_mutable_immutable_data_name.py
49 49_mutable_immutable_data_eggs.py
50 50_mutable_immutable_data_eggs_2.py
51 51_tuple_data_type.py
52 52_tuple_data_type_eggs.py
53 53_tuple_data_type_2.py
54 54_converting_types_list.py
55 55_references.py
56 56_references_spam.py
57 57_passing_references.py
58 58_passing_references_hello.py
59 59_copy_module_s.py
60 60_comma_code.py
61 61_character_picture_grid.py
62 62_character_picture_grid_oo.py

Topic: The List Data Type

01/62 01_list_data_type.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
[1, 2, 3]
```

```
# OUT:      [1, 2, 3]
```

```
['cat', 'bat', 'rat', 'elephant']
```

```
# OUT:      ['cat', 'bat', 'rat', 'elephant']
```

```
['hello', 3.1415, True, None, 42]
```

```
# OUT:      ['hello', 3.1415, True, None, 42]
```

```
spam = ['cat', 'bat', 'rat', 'elephant']
```

```
spam
```

```
# OUT:      ['cat', 'bat', 'rat', 'elephant']
```

Topic: Getting Individual Values in a List with Indexes

02/62 02_getting_individual_values.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = ['cat', 'bat', 'rat', 'elephant']
spam[0]
# OUT:      'cat'
spam[1]
# OUT:      'bat'
spam[2]
# OUT:      'rat'
spam[3]
# OUT:      'elephant'
['cat', 'bat', 'rat', 'elephant'][3]
# OUT:      'elephant'
'Hello ' + spam[0]
# OUT:      'Hello cat'
'The ' + spam[1] + ' ate the ' + spam[0] + '.'
# OUT:      'The bat ate the cat.'
```

03/62 03_getting_individual_values_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['cat', 'bat', 'rat', 'elephant']  
spam[10000]  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#9>", line 1, in <module>  
# OUT:     spam[10000]  
# OUT: IndexError: list index out of range
```

04/62 04_getting_individual_values_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

spam = ['cat', 'bat', 'rat', 'elephant']
spam[1]
# OUT: 'bat'
spam[1.0]
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#13>", line 1, in <module>
# OUT:     spam[1.0]
# OUT: TypeError: list indices must be integers, not float
spam[int(1.0)]
# OUT: 'bat'
```

05/62 05_getting_individual_values_spam_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = [['cat', 'bat'], [10, 20, 30, 40, 50]]  
spam[0]  
# OUT: ['cat', 'bat']  
spam[0][1]  
# OUT: 'bat'  
spam[1][4]  
# OUT: 50
```

Topic: Negative Indexes

06/62 06_negative_indexes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['cat', 'bat', 'rat', 'elephant']  
spam[-1]  
# OUT: 'elephant'  
spam[-3]  
# OUT: 'bat'  
'The ' + spam[-1] + ' is afraid of the ' + spam[-3] + '.'  
# OUT: 'The elephant is afraid of the bat.'
```

Topic: Getting Sublists with Slices

07/62 07_getting_sublists_slices.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['cat', 'bat', 'rat', 'elephant']  
spam[0:4]  
# OUT: ['cat', 'bat', 'rat', 'elephant']  
spam[1:3]  
# OUT: ['bat', 'rat']  
spam[0:-1]  
# OUT: ['cat', 'bat', 'rat']
```

08/62 08_getting_sublists_slices_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['cat', 'bat', 'rat', 'elephant']  
spam[:2]  
# OUT: ['cat', 'bat']  
spam[1:]  
# OUT: ['bat', 'rat', 'elephant']  
spam[:]  
# OUT: ['cat', 'bat', 'rat', 'elephant']
```

Topic: Getting a List's Length with len()

09/62 09_getting_list_s.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = ['cat', 'dog', 'moose']
```

```
len(spam)
```

```
# OUT: 3
```

Topic: Changing Values in a List with Indexes

10/62 10_changing_values_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = ['cat', 'bat', 'rat', 'elephant']  
>>> spam[1] = 'aardvark'  
>>> spam  
# OUT: ['cat', 'aardvark', 'rat', 'elephant']  
>>> spam[2] = spam[1]  
>>> spam  
# OUT: ['cat', 'aardvark', 'aardvark', 'elephant']  
>>> spam[-1] = 12345  
>>> spam  
# OUT: ['cat', 'aardvark', 'aardvark', 12345]
```

Topic: List Concatenation and List Replication

11/62 11_list_concatenation_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
[1, 2, 3] + ['A', 'B', 'C']
```

```
# OUT: [1, 2, 3, 'A', 'B', 'C']
```

```
['X', 'Y', 'Z'] * 3
```

```
# OUT: ['X', 'Y', 'Z', 'X', 'Y', 'Z', 'X', 'Y', 'Z']
```

```
spam = [1, 2, 3]
```

```
spam = spam + ['A', 'B', 'C']
```

```
spam
```

```
# OUT: [1, 2, 3, 'A', 'B', 'C']
```

Topic: Removing Values from Lists with del Statements

12/62 12_removing_values_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = ['cat', 'bat', 'rat', 'elephant']  
>>> del spam[2]  
>>> spam  
# OUT: ['cat', 'bat', 'elephant']  
>>> del spam[2]  
>>> spam  
# OUT: ['cat', 'bat']
```

Topic: Working with Lists

13/62 13_working_lists.py [script]

Script: save and run; predict the output first, then compare.

```
catName1 = 'Zophie'  
catName2 = 'Pooka'  
catName3 = 'Simon'  
catName4 = 'Lady Macbeth'  
catName5 = 'Fat-tail'  
catName6 = 'Miss Cleo'
```

14/62 14_working_lists_enter.py [script]

Script: save and run; predict the output first, then compare.

```
print('Enter the name of cat 1:')
catName1 = input()
print('Enter the name of cat 2:')
catName2 = input()
print('Enter the name of cat 3:')
catName3 = input()
print('Enter the name of cat 4:')
catName4 = input()
print('Enter the name of cat 5:')
catName5 = input()
print('Enter the name of cat 6:')
catName6 = input()
print('The cat names are:')
print(catName1 + ' ' + catName2 + ' ' + catName3 + ' ' + catName4 + ' ' +
catName5 + ' ' + catName6)
```

15/62 15_working_lists_catnames.py [script]

Script: save and run; predict the output first, then compare.

```
catNames = []
while True:
    print('Enter the name of cat ' + str(len(catNames) + 1) +
          ' (Or enter nothing to stop.):')
    name = input()
    if name == '':
        break
    catNames = catNames + [name] # list concatenation
print('The cat names are:')
for name in catNames:
    print(' ' + name)
```

16/62 16_working_lists_enter_2.py [script]

Script: save and run; predict the output first, then compare.

```
Enter the name of cat 1 (Or enter nothing to stop.):
Zophie
Enter the name of cat 2 (Or enter nothing to stop.):
Pooka
Enter the name of cat 3 (Or enter nothing to stop.):
Simon
Enter the name of cat 4 (Or enter nothing to stop.):
Lady Macbeth
Enter the name of cat 5 (Or enter nothing to stop.):
Fat-tail
Enter the name of cat 6 (Or enter nothing to stop.):
Miss Cleo
Enter the name of cat 7 (Or enter nothing to stop.):
```

The cat names are:

```
  Zophie
  Pooka
  Simon
  Lady Macbeth
  Fat-tail
  Miss Cleo
```

Topic: Using for Loops with Lists

17/62 17_using_loops_lists.py [script]

Script: save and run; predict the output first, then compare.

```
for i in range(4):  
    print(i)
```

18/62 18_using_loops_lists_2.py [script]

Script: save and run; predict the output first, then compare.

```
0  
1  
2  
3
```

19/62 19_using_loops_lists_3.py [script]

Script: save and run; predict the output first, then compare.

```
for i in [0, 1, 2, 3]:  
    print(i)
```

20/62 20_using_loops_lists_supplies.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
supplies = ['pens', 'staplers', 'flame-throwers', 'binders']  
for i in range(len(supplies)):  
# OUT:      print('Index ' + str(i) + ' in supplies is: ' + supplies[i])  
  
# OUT: Index 0 in supplies is: pens  
# OUT: Index 1 in supplies is: staplers  
# OUT: Index 2 in supplies is: flame-throwers  
# OUT: Index 3 in supplies is: binders
```

Topic: The in and not in Operators

21/62 21_not_operators.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'howdy' in ['hello', 'hi', 'howdy', 'heyas']  
# OUT: True  
spam = ['hello', 'hi', 'howdy', 'heyas']  
'cat' in spam  
# OUT: False  
'howdy' not in spam  
# OUT: False  
'cat' not in spam  
# OUT: True
```

22/62 22_not_operators_mypets.py [script]

Script: save and run; predict the output first, then compare.

```
myPets = ['Zophie', 'Pooka', 'Fat-tail']
print('Enter a pet name:')
name = input()
if name not in myPets:
    print('I do not have a pet named ' + name)
else:
    print(name + ' is my pet.')
```

23/62 23_not_operators_enter.py [script]

Script: save and run; predict the output first, then compare.

```
Enter a pet name:
```

```
Footfoot
```

```
I do not have a pet named Footfoot
```

Topic: The Multiple Assignment Trick

24/62 24_multiple_assignment_trick.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
cat = ['fat', 'orange', 'loud']
size = cat[0]
color = cat[1]
disposition = cat[2]
```

25/62 25_multiple_assignment_trick_cat.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
cat = ['fat', 'orange', 'loud']
```

```
size, color, disposition = cat
```

26/62 26_multiple_assignment_trick_cat_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
cat = ['fat', 'orange', 'loud']  
size, color, disposition, name = cat  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#84>", line 1, in <module>  
# OUT:     size, color, disposition, name = cat  
# OUT: ValueError: need more than 3 values to unpack
```

27/62 27_multiple_assignment_trick_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> a, b = 'Alice', 'Bob'  
>>> a, b = b, a  
>>> print(a)  
# OUT: 'Bob'  
>>> print(b)  
# OUT: 'Alice'
```

Topic: Augmented Assignment Operators

28/62 28_augmented_assignment_operators.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 42
```

```
spam = spam + 1
```

```
spam
```

```
# OUT: 43
```

29/62 29_augmented_assignment_operators_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 42
```

```
spam += 1
```

```
spam
```

```
# OUT: 43
```

30/62 30_augmented_assignment_operators_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = 'Hello'  
spam += ' world!'  
spam  
# OUT: 'Hello world!'
```

31/62 31_augmented_assignment_operators_bacon.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
bacon = ['Zophie']
```

```
bacon *= 3
```

```
bacon
```

```
# OUT: ['Zophie', 'Zophie', 'Zophie']
```

Topic: Finding a Value in a List with the index() Method

32/62 32_finding_value_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
>>> spam = ['hello', 'hi', 'howdy', 'heyas']  
>>> spam.index('hello')  
# OUT: 0  
>>> spam.index('heyas')  
# OUT: 3  
>>> spam.index('howdy howdy howdy')  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#31>", line 1, in <module>  
# OUT:     spam.index('howdy howdy howdy')  
# OUT: ValueError: 'howdy howdy howdy' is not in list
```

33/62 33_finding_value_list_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['Zophie', 'Pooka', 'Fat-tail', 'Pooka']  
spam.index('Pooka')  
# OUT: 1
```

Topic: Adding Values to Lists with the append() and insert() Methods

34/62 34_adding_values_lists.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
>>> spam = ['cat', 'dog', 'bat']
```

```
>>> spam.append('moose')
```

```
>>> spam
```

```
# OUT: ['cat', 'dog', 'bat', 'moose']
```

35/62 35_adding_values_lists_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = ['cat', 'dog', 'bat']  
spam.insert(1, 'chicken')  
spam  
# OUT: ['cat', 'chicken', 'dog', 'bat']
```

36/62 36_adding_values_lists_eggs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

eggs = 'hello'
eggs.append('world')
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#19>", line 1, in <module>
# OUT:     eggs.append('world')
# OUT: AttributeError: 'str' object has no attribute 'append'
bacon = 42
bacon.insert(1, 'world')
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#22>", line 1, in <module>
# OUT:     bacon.insert(1, 'world')
# OUT: AttributeError: 'int' object has no attribute 'insert'
```

Topic: Removing Values from Lists with remove()

37/62 37_removing_values_from_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = ['cat', 'bat', 'rat', 'elephant']
```

```
spam.remove('bat')
```

```
spam
```

```
# OUT: ['cat', 'rat', 'elephant']
```

38/62 38_removing_values_from_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = ['cat', 'bat', 'rat', 'elephant']  
spam.remove('chicken')  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#11>", line 1, in <module>  
# OUT:     spam.remove('chicken')  
# OUT: ValueError: list.remove(x): x not in list
```

39/62 39_removing_values_from_spam_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = ['cat', 'bat', 'rat', 'cat', 'hat', 'cat']  
spam.remove('cat')  
spam  
# OUT: ['bat', 'rat', 'cat', 'hat', 'cat']
```

Topic: Sorting the Values in a List with the sort() Method

40/62 40_sorting_values_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
>>> spam = [2, 5, 3.14, 1, -7]
```

```
>>> spam.sort()
```

```
>>> spam
```

```
# OUT: [-7, 1, 2, 3.14, 5]
```

```
>>> spam = ['ants', 'cats', 'dogs', 'badgers', 'elephants']
```

```
>>> spam.sort()
```

```
>>> spam
```

```
# OUT: ['ants', 'badgers', 'cats', 'dogs', 'elephants']
```

41/62 41_sorting_values_list_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam.sort(reverse=True)  
spam  
# OUT: ['elephants', 'dogs', 'cats', 'badgers', 'ants']
```

42/62 42_sorting_values_list_spam_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = [1, 3, 2, 4, 'Alice', 'Bob']  
spam.sort()  
# OUT: Traceback (most recent call last):  
# OUT: File "<pyshell#70>", line 1, in <module>  
# OUT:     spam.sort()  
# OUT: TypeError: unorderable types: str() < int()
```

43/62 43_sorting_values_list_spam_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
spam = ['Alice', 'ants', 'Bob', 'badgers', 'Carol', 'cats']  
spam.sort()  
spam  
# OUT: ['Alice', 'Bob', 'Carol', 'ants', 'badgers', 'cats']
```

44/62 44_sorting_values_list_spam_4.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = ['a', 'z', 'A', 'Z']
```

```
spam.sort(key=str.lower)
```

```
spam
```

```
# OUT: ['a', 'A', 'z', 'Z']
```

Topic: Example Program: Magic 8 Ball with a List

45/62 45_example_program_magic.py [script]

Script: save and run; predict the output first, then compare.

```
import random

messages = ['It is certain',
            'It is decidedly so',
            'Yes definitely',
            'Reply hazy try again',
            'Ask again later',
            'Concentrate and ask again',
            'My reply is no',
            'Outlook not so good',
            'Very doubtful']

print(messages[random.randint(0, len(messages) - 1)])
```

Topic: List-like Types: Strings and Tuples

46/62 46_list_like_types.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
name = 'Zophie'
name[0]
# OUT: 'Z'
name[-2]
# OUT: 'i'
name[0:4]
# OUT: 'Zoph'
'Zo' in name
# OUT: True
'z' in name
# OUT: False
'p' not in name
# OUT: False
for i in name:
# OUT:      print('* * * ' + i + ' * * *')

# OUT: * * * Z * * *
# OUT: * * * o * * *
# OUT: * * * p * * *
# OUT: * * * h * * *
# OUT: * * * i * * *
# OUT: * * * e * * *
```

Topic: Mutable and Immutable Data Types

47/62 47_mutable_immutable_data.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
name = 'Zophie a cat'  
name[7] = 'the'  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#50>", line 1, in <module>  
# OUT:     name[7] = 'the'  
# OUT: TypeError: 'str' object does not support item assignment
```

48/62 48_mutable_immutable_data_name.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
name = 'Zophie a cat'  
newName = name[0:7] + 'the' + name[8:12]  
name  
# OUT: 'Zophie a cat'  
newName  
# OUT: 'Zophie the cat'
```

49/62 49_mutable_immutable_data_eggs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
eggs = [1, 2, 3]
```

```
eggs = [4, 5, 6]
```

```
eggs
```

```
# OUT: [4, 5, 6]
```

50/62 50_mutable_immutable_data_eggs_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
eggs = [1, 2, 3]  
del eggs[2]  
del eggs[1]  
del eggs[0]  
eggs.append(4)  
eggs.append(5)  
eggs.append(6)  
eggs  
# OUT: [4, 5, 6]
```

Topic: The Tuple Data Type

51/62 51_tuple_data_type.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
eggs = ('hello', 42, 0.5)
```

```
eggs[0]
```

```
# OUT: 'hello'
```

```
eggs[1:3]
```

```
# OUT: (42, 0.5)
```

```
len(eggs)
```

```
# OUT: 3
```

52/62 52_tuple_data_type_eggs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
eggs = ('hello', 42, 0.5)  
eggs[1] = 99  
# OUT: Traceback (most recent call last):  
# OUT:   File "<pyshell#5>", line 1, in <module>  
# OUT:     eggs[1] = 99  
# OUT: TypeError: 'tuple' object does not support item assignment
```

53/62 53_tuple_data_type_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
type(('hello',))  
# OUT: <class 'tuple'>  
type('hello')  
# OUT: <class 'str'>
```

Topic: Converting Types with the list() and tuple() Functions

54/62 54_converting_types_list.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
>>> tuple(['cat', 'dog', 5])
```

```
# OUT: ('cat', 'dog', 5)
```

```
>>> list(('cat', 'dog', 5))
```

```
# OUT: ['cat', 'dog', 5]
```

```
>>> list('hello')
```

```
# OUT: ['h', 'e', 'l', 'l', 'o']
```

Topic: References

55/62 55_references.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = 42
cheese = spam
spam = 100
spam
# OUT: 100
cheese
# OUT: 42
```

56/62 56_references_spam.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
spam = [0, 1, 2, 3, 4, 5]  
cheese = spam  
cheese[1] = 'Hello!'  
spam  
# OUT:      [0, 'Hello!', 2, 3, 4, 5]  
cheese  
# OUT:      [0, 'Hello!', 2, 3, 4, 5]
```

Topic: Passing References

57/62 57_passing_references.py [script]

Script: save and run; predict the output first, then compare.

```
def eggs(someParameter):  
    someParameter.append('Hello')  
  
spam = [1, 2, 3]  
eggs(spam)  
print(spam)
```

58/62 58_passing_references_hello.py [script]

Script: save and run; predict the output first, then compare.

```
[1, 2, 3, 'Hello']
```

Topic: The copy Module's copy() and deepcopy() Functions

59/62 59_copy_module_s.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import copy
spam = ['A', 'B', 'C', 'D']
cheese = copy.copy(spam)
cheese[1] = 42
spam
# OUT: ['A', 'B', 'C', 'D']
cheese
# OUT: ['A', 42, 'C', 'D']
```

Topic: Comma Code

60/62 60_comma_code.py [script]

Script: save and run; predict the output first, then compare.

```
spam = ['apples', 'bananas', 'tofu', 'cats']
```

Topic: Character Picture Grid

61/62 61_character_picture_grid.py [script]

Script: save and run; predict the output first, then compare.

```
grid = [['.', '.', '.', '.', '.', '.'],
        ['.', '0', '0', '.', '.', '.'],
        ['0', '0', '0', '0', '.', '.'],
        ['0', '0', '0', '0', '0', '.'],
        ['.', '0', '0', '0', '0', '0'],
        ['0', '0', '0', '0', '0', '.'],
        ['0', '0', '0', '0', '.', '.'],
        ['.', '0', '0', '.', '.', '.'],
        ['.', '.', '.', '.', '.', '.']]
```

62/62 62_character_picture_grid_oo.py [script]

Script: save and run; predict the output first, then compare.

```
..00.00..  
.0000000.  
.0000000.  
..00000..  
...000...  
....0....
```